$$\textsc{Float-Const}\ \frac{\mathsf{has}(V, c)}{\Gamma \vdash c : \mathbf{float}[B; V]} \qquad \textsc{Continuous-1}\ \frac{\Gamma \vdash e : \mathbf{float}[B_1; V_1] \quad \mathsf{recovers}(B_1, V_1) \quad \mathsf{dispatchable}(B_1, V_1)}{\Gamma \vdash \mathbf{exponential}(e) : \mathbf{float}[B; \top]}$$

$$\textsc{Continuous-2}\ \frac{\begin{array}{lll}\Gamma \vdash e_1 : \mathbf{float}[B_1; V_1] & \mathsf{recovers}(B_1, V_1) & \mathsf{dispatchable}(B_1, V_1)\\ \Gamma \vdash e_2 : \mathbf{float}[B_2; V_2] & \mathsf{recovers}(B_2, V_2) & \mathsf{dispatchable}(B_2, V_2)\end{array}}{\Gamma \vdash \mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]}$$

$$\textsc{Less}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \quad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \quad \mathsf{answers}_{<}(B, V_1, V_2)}{\Gamma \vdash e_1 < e_2 : \mathbf{bool}}$$

$$\textsc{Less-Equal}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \quad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \quad \mathsf{answers}_{\leq}(B, V_1, V_2)}{\Gamma \vdash e_1 \leq e_2 : \mathbf{bool}}$$

Fig. 6. Typing rules for float types and continuous distributions (**uniform** is a representative for all other two-argument distributions). The notions *recovers* and *answers* are defined in Definitions 3.2, 3.3 and 3.4. Here Γ is a typing environment of the form $\{x_1 : \tau_1, \ldots, x_n : \tau_n\}$, where $\Gamma, x : \tau_1 = (\Gamma \setminus \{x : \tau \mid \tau \text{ type}\}) \cup \{x : \tau_1\}$.

3.3 Typing Rules

The typing rules are split into three parts: Rules specific to float types and continuous distributions (Figure 6), standard rules for all other language constructs (Figure 7), and subtyping (Figure 8).

To understand the float typing rules, it is helpful to understand how discretization works. A type **float**[B; V] is discretized as follows: If $B \neq \top$ is a finite set of cuts, then the type is discretized to a type $\mathbf{fin}(n+1)$ where $n+1 = |\mathsf{Intervals}(B)|$ is the number of intervals induced by B. For instance, if $B = \{<0.5, \leq 0.8\}$, then the type is discretized to $\mathbf{fin}(3)$, where $0_{\#3}$ represents the interval $(-\infty, 0.5)$, $1_{\#3}$ represents the interval $[0.5, 0.8]$, and $2_{\#3}$ represents the interval $(0.8, \infty)$. If $B = \top$, then no discretization is performed, and the type remains **float**.

To understand the typing rules, keep in mind that the cut sets B need to be sufficiently fine-grained — i.e., contain enough cuts — so that the discretized program has enough information to determine how the original continuous program behaved.

Constants. Rule Float-Const assigns a constant c the type $\mathbf{float}[B; V]$ whenever $\mathsf{has}(V, c)$ holds (i.e., $c \in V$ or $V = \top$). Constants do not constrain the cut set, but recording them in V ensures later comparisons know which values must remain distinguishable.

Continuous distributions. Rule Continuous-1 types a single-argument distribution when its argument e has type $\mathbf{float}[B; V]$, $\mathsf{recovers}(B, V)$, and $\mathsf{dispatchable}(B, V)$:

Definition 3.2. Let $B \in \mathcal{B}$ and $V \in \mathcal{V}$. We say that $\mathsf{recovers}(B, V)$ if $B = \top$ or ($V \neq \top$ and every interval I induced by B contains at most one value in V, i.e., $\forall I \in \mathsf{Intervals}(B) : |V \cap I| \leq 1$).

Intuitively, $\mathsf{recovers}(B, V)$ means that the discretization induced by B is lossless on V: from the resulting interval index, the original value in V can be uniquely recovered. This injectivity property is vacuously true for $V = \emptyset$, so distribution parameters also require $\mathsf{dispatchable}(B, V)$, defined by $B = \top$ or $V \notin \{\emptyset, \top\}$. When $B \neq \top$, the value set is therefore finite and nonempty, and B separates its values. The separate dispatchability premise ensures that the distribution-dispatch transformation has at least one case. §5 revisits these invariants. The rule for two-argument distributions is analogous.

Comparisons. For $e < c$ with $e : \mathbf{float}[B_1; \top]$ and $c : \mathbf{float}[B_2; \{c\}]$, the discretized e must still decide the predicate, so B_1 must include $<c$. If c can take any value in V_2, every $<v$ for $v \in V_2$ must appear. Symmetrically, for $c < e$ with $e : \mathbf{float}[B_2; V_2]$, B_2 must contain $\leq c$ so the discretized

$$\textsc{Sub-Refl}\ \frac{\tau \in \{\texttt{int}, \texttt{bool}, \texttt{fin}(n)\}}{\tau <: \tau} \qquad \textsc{Sub-Float}\ \frac{B_1 = B_2 \qquad V_1 \sqsubseteq V_2}{\texttt{float}[B_1; V_1] <: \texttt{float}[B_2; V_2]} \qquad \textsc{Sub-Pair}\ \frac{\tau_1 <: \tau_1' \qquad \tau_2 <: \tau_2'}{\tau_1 * \tau_2 <: \tau_1' * \tau_2'}$$

$$\textsc{Sub-List}\ \frac{\tau <: \tau'}{\texttt{list}(\tau) <: \texttt{list}(\tau')} \qquad \textsc{Sub-Arrow}\ \frac{\tau_1' <: \tau_1 \qquad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'} \qquad \textsc{Sub-Type}\ \frac{\tau_1 <: \tau_2 \qquad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2}$$

Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

can also be considered to range over a superset of those values. For B, we require equality to make sure that the discretized representation of the expression is not changed by subtyping.

All other typing rules are standard and depicted in Figure 7 and §D, respectively.

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with its most precise type. Specifically, we focus on expressions of type $\texttt{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed expressions, we use a two-phase approach: the first phase traverses the program once to generate constraints over cut and value sets; the second phase solves those constraints to produce a fully annotated program.

The distinction between typing and type inference is important. The correctness argument cleanly separates into two pieces: (1) type inference produces a typing derivation matching our typing rules, and (2) given such a typing derivation, discretization is semantics preserving. Thus the proof of discretization does not have to reason directly about the type inference procedure.

4.1 Constraint Generation

We use the constraint language in Figure 9 to express requirements over the type structure and float refinements. We adopt the framework of [38], introducing a new judgment of the form $\Gamma \vdash e : \tau \mid C$, which reads as *in the environment* Γ, *e has type* τ *and produces the set of constraints* C. In this judgment, Γ and e are inputs to the type inference algorithm, while τ and C are outputs: the algorithm infers the type τ for expression e and generates the constraints C that must hold for this typing to be valid. The inference rules formalize which constraints must be generated and later solved for a given expression e to have inferred type τ. Figure 10 gives the constraint generation rules for Slice. We focus on a small but representative subset of constructs in Slice, capturing the key ideas behind constraint generation for the *full* language.

The type inference rules closely mirror the typing rules in Figure 6, with two key differences. First, the inference rules generate a constraint set instead of having those constraints appear in the premises. Second, subtyping is handled in a syntax-directed way: rather than using a general subtyping rule, we bake the subtyping relation into each rule.

Example 4.1. To make the constraint generation process concrete, Figure 11 walks through §2.3. Each subexpression is annotated with type variables for the cut sets (B_i) and value sets (V_i), and the constraints produced at each step are shown explicitly.

4.2 Constraint Solving

Once constraint generation completes, we obtain a system of constraints over type variables, cut sets B, and value sets V. The constraint-solving phase proceeds in two steps:

$$C ::= \tau_1 <: \tau_2 \mid \mathsf{recovers}(B, V) \mid \mathsf{dispatchable}(B, V) \mid \mathsf{answers}_<(B, V_1, V_2) \mid \mathsf{answers}_\leq(B, V_1, V_2) \mid \mathsf{has}(V, c)$$
$$\mid B_1 \sqsubseteq B_2 \mid B_1 = B_2 \mid V_1 \sqsubseteq V_2 \mid V_1 = V_2 \mid B_1 \equiv_\top B_2 \mid C_1, C_2$$

Fig. 9. Syntax for our constraint language. Constraints correspond to the semantic requirements in our type system: subtyping ensures type compatibility, $\mathsf{recovers}(B, V)$ ensures that distribution parameters can be recovered after discretization, $\mathsf{dispatchable}(B, V)$ ensures that a finite dispatch has at least one case, $\mathsf{answers}_<(B, V_1, V_2)$ and $\mathsf{answers}_\leq(B, V_1, V_2)$ ensure that comparison results can be determined after discretization, and $\mathsf{has}(V, c)$ ensures that constants are properly tracked.

$$\textsc{TFloat-Const}\ \frac{}{\Gamma \vdash c : \mathbf{float}[B; V] \mid \mathsf{has}(V, c)} \qquad \textsc{TIf}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \mid C_1 \qquad \Gamma \vdash e_2 : \tau_2 \mid C_2 \qquad \Gamma \vdash e_3 : \tau_3 \mid C_3}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau_4 \mid C_1, C_2, C_3, \tau_2 <: \tau_4, \tau_3 <: \tau_4}$$

$$\textsc{TVar}\ \frac{}{\Gamma, x : \tau \vdash x : \tau \mid \emptyset} \qquad \textsc{TLess}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \mid C_1 \qquad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \mid C_2}{\Gamma \vdash e_1 < e_2 : \mathbf{bool} \mid C_1, C_2, \mathsf{answers}_<(B, V_1, V_2)}$$

$$\textsc{TContinuous-1}\ \frac{\Gamma \vdash e : \mathbf{float}[B_1; V_1] \mid C}{\Gamma \vdash \mathbf{exponential}(e) : \mathbf{float}[B; V] \mid C, \mathsf{recovers}(B_1, V_1), \mathsf{dispatchable}(B_1, V_1), B_1 \equiv_\top B, V = \top}$$

$$\textsc{TContinuous-2}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B_1; V_1] \mid C_1 \qquad \Gamma \vdash e_2 : \mathbf{float}[B_2; V_2] \mid C_2}{\Gamma \vdash \mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; V] \mid C_1, C_2, \mathsf{recovers}(B_1, V_1), \mathsf{dispatchable}(B_1, V_1), B_1 \equiv_\top B, \mathsf{recovers}(B_2, V_2), \mathsf{dispatchable}(B_2, V_2), B_2 \equiv_\top B, V = \top}$$

$$\textsc{TLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \mid C_1 \qquad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \mid C_2}{\Gamma \vdash e_1 \leq e_2 : \mathbf{bool} \mid C_1, C_2, \mathsf{answers}_\leq(B, V_1, V_2)} \qquad \textsc{TLet}\ \frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \mid C_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2 \mid C_1, C_2}$$

Fig. 10. Constraint generation rules for type inference for float-typed expressions.

(1) **Subtyping reduction**: We first process all subtyping constraints of the form $\tau_1 <: \tau_2$. Since subtyping for float types follows the rule $\mathbf{float}[B_1; V_1] <: \mathbf{float}[B_2; V_2]$ if and only if $B_1 = B_2$ and $V_1 \sqsubseteq V_2$, we reduce each subtyping constraint to corresponding equality and subset constraints on the cut and value sets. Subtyping elimination continues until only constraints among B's and V's remain.
(2) **Lattice constraint solving**: After subtyping reduction, we are left with constraints purely over cut sets and value sets: equalities, subset relations, and the specialized constraints $\mathsf{recovers}(B, V)$, $\mathsf{dispatchable}(B, V)$, $\mathsf{answers}_<(B, V_1, V_2)$, and $\mathsf{has}(V, c)$. These constraints are solved using a monotone fixed-point algorithm over the cut and value lattices.

Subtyping Reduction. Subtyping reduction eliminates all remaining type-level subtyping constraints by recursively decomposing them according to type structure. For base float types, a constraint $\mathbf{float}[B_1; V_1] <: \mathbf{float}[B_2; V_2]$ is reduced to the conjunction $B_1 = B_2$ and $V_1 \sqsubseteq V_2$. For compound types, subtyping is handled structurally before reaching floats. For example, a constraint $\mathbf{float}[B_1; V_1] * \mathbf{float}[B_2; V_2] <: \mathbf{float}[B_5; V_5] * \mathbf{float}[B_6; V_6]$ is decomposed into $\mathbf{float}[B_1; V_1] <: \mathbf{float}[B_5; V_5]$ and $\mathbf{float}[B_2; V_2] <: \mathbf{float}[B_6; V_6]$, each of which is then reduced to equality and subset constraints on the corresponding cut and value sets. This process is syntactic and continues until no subtyping constraints remain, leaving only constraints over B and V.

Program	Constraints Emitted
`let x = if flip()`	
`then uniform(0 :float`$[B_1; V_1]$`,`	+ has$(V_1, 0)$
`2 :float`$[B_2; V_2]$`)`	+ has$(V_2, 2)$
`:float`$[B_3; V_3]$	+ recovers(B_1, V_1), dispatchable(B_1, V_1), $B_1 \equiv_\top B_3$, recovers(B_2, V_2), dispatchable(B_2, V_2), $B_2 \equiv_\top B_3, V_3 = \top$
`else gaussian(0 :float`$[B_4; V_4]$`,`	+ has$(V_4, 0)$
`1 :float`$[B_5; V_5]$`)`	+ has$(V_5, 1)$
`:float`$[B_6; V_6]$	+ recovers(B_4, V_4), dispatchable(B_4, V_4), $B_4 \equiv_\top B_6$, recovers(B_5, V_5), dispatchable(B_5, V_5), $B_5 \equiv_\top B_6, V_6 = \top$
`:float`$[B_7; V_7]$	+ `float`$[B_3; V_3]$ <: `float`$[B_7; V_7]$, `float`$[B_6; V_6]$ <: `float`$[B_7; V_7]$
`in`	
`let y = if x :float`$[B_7; V_7]$	+ $\emptyset$
`<`	
`1.5 :float`$[B_8; V_8]$	+ has$(V_8, 1.5)$
`:bool`	+ answers$_<(B_7, V_7, V_8), B_7 = B_8$
`then 1.8 :float`$[B_9; V_9]$	+ has$(V_9, 1.8)$
`else 0.3 :float`$[B_{10}; V_{10}]$	+ has$(V_{10}, 0.3)$
`:float`$[B_{11}; V_{11}]$	+ `float`$[B_9; V_9]$ <: `float`$[B_{11}; V_{11}]$, `float`$[B_{10}; V_{10}]$ <: `float`$[B_{11}; V_{11}]$
`in`	
`x :float`$[B_7; V_7]$	+ $\emptyset$
`<=`	
`y :float`$[B_{11}; V_{11}]$	+ $\emptyset$
`:bool`	+ answers$_\leq(B_7, V_7, V_{11}), B_7 = B_{11}$

Fig. 11. Program from §2.3 (left) and its emitted constraints from each line (right) from TFloat-Const, TContinuous-2, TIf, TVar, and TLess and TLessEq following the constraint generation rules in Figure 10. Lines annotated with $\emptyset$ emit no constraints.

Lattice Solving. The lattice solver keeps track of the current contents of all B and V variables. Initially, all B and V variables are set to $\emptyset$. During the solving process, the variables may go up in the lattice to a finite set, or to $\top$. Each of the constraints is handled as follows:

- **Value containment** (has(V, c)): We update V to include the constant c, i.e., $V := V \sqcup \{c\}$. If V is already $\top$, the constraint is trivially satisfied. (1)
- **Top assignment** ($V = \top$): We immediately set V to $\top$, representing that the value set contains all possible float values. (2)
- **Subset constraints** ($B_1 \sqsubseteq B_2$ or $V_1 \sqsubseteq V_2$): We ensure that the contents of the first variable are always a subset of the second. When B_1 or V_1 is updated, we propagate the changes to B_2 or V_2 by taking the least upper bound. (3)

- **Recoverability and dispatchability** (recovers(B, V) and dispatchable(B, V)): Recoverability ensures that the cut set B can distinguish all values in V. If the cuts do not separate the values in the current solver state, we insert additional cuts to satisfy this condition. There is no canonical way to do this, so we use a heuristic: we insert a cut $<x$ for each value $x \in V$ that is not already separated by existing cuts. Dispatchability separately requires a finite, nonempty V when $B \neq \top$. Because $V = \emptyset$ causes no propagating update, the implementation discharges these logical obligations after monotone propagation reaches its fixed point by traversing the typed sampler nodes and validating their final cut and value bags. Every public discretization entry point performs this validation and rejects an empty or infinite dispatch domain instead of falling back to a sampler. If $B = \top$, both constraints are trivially satisfied. (4)
- **Less-than answerability** (answers$_<(B, V_1, V_2)$): Intuitively, this constraint ensures that B contains enough cuts to answer all possible comparisons of the form "$v_1 < v_2$" where $v_1 \in V_1$ and $v_2 \in V_2$. If $B = \top$, the constraint is trivially satisfied. Otherwise, we again have a heuristic choice: we add a cut $<v$ for each value $v \in V_2$ (if $V_2 \neq \top$) and $\leq v$ for each value $v \in V_1$ (if $V_1 \neq \top$). If V_1 and V_2 are both $\top$, we set B to $\top$. (5)
- **Less-equal answerability** (answers$_\leq(B, V_1, V_2)$): Analogous to less-than. (5)
- **Equality constraints** ($B_1 = B_2$ or $V_1 = V_2$): We unify the variables, treating them as the same variable throughout the solving process. Any update to one variable is immediately reflected in all variables unified with it. (6)

The solver repeatedly processes constraints until reaching a fixed point, where no constraint causes any further updates to the variables. The monotonicity of the lattice operations ensures that this process terminates. This procedure may add more cuts to Bs than strictly necessary, but it never sets $B = \top$ unless that is the only way to satisfy all constraints. In other words, the discretization itself may be suboptimal in terms of the number of cuts.

Lattice constraint solving takes time polynomial in the size of the program: at worst, *every* constant in the program gets added to the cut set or value set of every subexpression of type $\texttt{float}[B; V]$.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., $\texttt{float}[B; V] \to \texttt{float}[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.2. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression e : **bool** and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \texttt{bool}]\!]$: **bool** such that e and $\mathcal{D}[\![e : \texttt{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. Exactness here is the equality proved by Theorem 6.1: the transformed program preserves the relevant probabilities of the source program, not merely an approximation with an error bound. Intuitively, when a program uses continuous quantities only

Subtyping Constraint	After Subtyping Reduction
$\texttt{float}[B_3; V_3] <: \texttt{float}[B_7; V_7]$	$B_3 = B_7,\ V_3 \sqsubseteq V_7$
$\texttt{float}[B_6; V_6] <: \texttt{float}[B_7; V_7]$	$B_6 = B_7,\ V_6 \sqsubseteq V_7$
$\texttt{float}[B_9; V_9] <: \texttt{float}[B_{11}; V_{11}]$	$B_9 = B_{11},\ V_9 \sqsubseteq V_{11}$
$\texttt{float}[B_{10}; V_{10}] <: \texttt{float}[B_{11}; V_{11}]$	$B_{10} = B_{11},\ V_{10} \sqsubseteq V_{11}$

(a) *Subtyping reduction.* Each subtyping constraint over float types is decomposed into an equality constraint on cut sets and a subset constraint on value sets.

	Constraints Processed	Solver Updates
		$\forall i\colon B_i = \emptyset, V_i = \emptyset$
①	$\text{has}(V_1, 0)$	$V_1 := V_1 \sqcup \{0\} = \{0\}$
	$\text{has}(V_2, 2)$	$V_2 := V_2 \sqcup \{2\} = \{2\}$
	$\text{has}(V_4, 0)$	$V_4 := V_4 \sqcup \{0\} = \{0\}$
	$\text{has}(V_5, 1)$	$V_5 := V_5 \sqcup \{1\} = \{1\}$
	$\text{has}(V_8, 1.5)$	$V_8 := V_8 \sqcup \{1.5\} = \{1.5\}$
	$\text{has}(V_9, 1.8)$	$V_9 := V_9 \sqcup \{1.8\} = \{1.8\}$
	$\text{has}(V_{10}, 0.3)$	$V_{10} := V_{10} \sqcup \{0.3\} = \{0.3\}$
②	$V_3 = \top$	$V_3 := \top$
	$V_6 = \top$	$V_6 := \top$
③	$V_3 \sqsubseteq V_7$	$V_7 := V_7 \sqcup \top = \top$
	$V_6 \sqsubseteq V_7$	$\emptyset$
	$V_9 \sqsubseteq V_{11}$	$V_{11} := V_{11} \sqcup \{1.8\} = \{1.8\}$
	$V_{10} \sqsubseteq V_{11}$	$V_{11} := V_{11} \sqcup \{0.3\} = \{0.3, 1.8\}$
④	$\text{recovers}(B_1, V_1),\ \text{dispatchable}(B_1, V_1)$	$\emptyset$
	$\text{recovers}(B_2, V_2),\ \text{dispatchable}(B_2, V_2)$	$\emptyset$
	$\text{recovers}(B_4, V_4),\ \text{dispatchable}(B_4, V_4)$	$\emptyset$
	$\text{recovers}(B_5, V_5),\ \text{dispatchable}(B_5, V_5)$	$\emptyset$
⑤	$\text{answers}_<(B_7, V_7, V_8)$	$B_7 := B_7 \sqcup \{<1.5\} = \{<1.5\}$
	$\text{answers}_\leq(B_7, V_7, V_{11})$	$B_7 := B_7 \sqcup \{\leq 0.3, \leq 1.8\}$ $= \{<1.5, \leq 0.3, \leq 1.8\}$
⑥	$B_7 = B_8, B_7 = B_{11}$	$B_7 = B_8, B_7 = B_{11}$
	$B_3 = B_7, B_6 = B_7$	$B_3 = B_7, B_6 = B_7$
	$B_9 = B_{10}, B_{10} = B_{11}$	$B_9 = B_{11}, B_{10} = B_{11}$

(b) *Lattice solving.* Each pass performs monotone updates to B_i or V_i. Lines annotated with $\emptyset$ denote constraint is trivially satisfied.

Cut Sets

$B_1 = \emptyset$
$B_2 = \emptyset$
$B_4 = \emptyset$
$B_5 = \emptyset$
$B_3 = B_6$
$= B_7 = B_8$
$= B_9 = B_{10}$
$= B_{11}$
$= \{<1.5, \leq 0.3, \leq 1.8\}$

Value Sets

$V_1 = \{0\}$
$V_2 = \{2\}$
$V_3 = \top$
$V_4 = \{0\}$
$V_5 = \{1\}$
$V_6 = \top$
$V_7 = \top$
$V_8 = \{1.5\}$
$V_9 = \{1.8\}$
$V_{10} = \{0.3\}$
$V_{11} = \{0.3, 1.8\}$

(c) Final cut sets and value sets obtained.

Fig. 12. Following Figure 11, the constraint solving process for §2.3.

$e : \tau$	Condition	$\mathcal{D}[\![e : \tau]\!]$
$x : \tau$		x
$c : \mathbf{float}[B; V]$	$B = \top$	c
	$B \neq \top$	$k_{\#(\lvert B\rvert+1)}$ such that $c \in \text{Intervals}(B)_k$
$\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2 : \tau_2$		$\mathbf{let}\ x = \mathcal{D}[\![e_1 : \tau_1]\!]\ \mathbf{in}\ \mathcal{D}[\![e_2 : \tau_2]\!]$
$(\mathbf{if}\ e_1 : \mathbf{bool}\ \mathbf{then}\ e_2 : \tau\ \mathbf{else}\ e_3 : \tau) : \tau$		$\mathbf{if}\ \mathcal{D}[\![e_1 : \mathbf{bool}]\!]\ \mathbf{then}\ \mathcal{D}[\![e_2 : \tau]\!]\ \mathbf{else}\ \mathcal{D}[\![e_3 : \tau]\!]$
$(e_1 : \mathbf{float}[B; V_1] < e_2 : \mathbf{float}[B; V_2]) : \mathbf{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \mathbf{float}[B; V_1]]\!] <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \mathbf{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \mathbf{float}[B; V_1]]\!] < \mathcal{D}[\![e_2 : \mathbf{float}[B; V_2]]\!]$
$(e_1 : \mathbf{float}[B; V_1] \leq e_2 : \mathbf{float}[B; V_2]) : \mathbf{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \mathbf{float}[B; V_1]]\!] \leq_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \mathbf{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \mathbf{float}[B; V_1]]\!] \leq \mathcal{D}[\![e_2 : \mathbf{float}[B; V_2]]\!]$
$\mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1],$ $e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B; V]$	$e_1, e_2 \in \mathbb{R},\ e_1 > e_2$	$\mathbf{diverge} : \tau$
	$B = \top$	$\mathbf{uniform}(\mathcal{D}[\![e_1 : \mathbf{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \mathbf{float}[B_2; V_2]]\!])$
	$e_1, e_2 \in \mathbb{R}$, $B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$	$\mathbf{discrete}(p_0, \ldots, p_n)$ where $p_k = \text{CDF}(b_{k+1}) - \text{CDF}(b_k)$
	$B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, $V_1 = \{v_1, \ldots, v_n\},\ n \geq 1$, $V_2 = \{w_1, \ldots, w_m\},\ m \geq 1$	$\mathbf{let}\ v_1' = \mathcal{D}[\![e_1 : \mathbf{float}[B_1; V_1]]\!]\ \mathbf{in}$ $\mathbf{let}\ v_2' = \mathcal{D}[\![e_2 : \mathbf{float}[B_2; V_2]]\!]\ \mathbf{in}$ $\mathbf{if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_1 : \mathbf{float}[B_1; V_1]]\!]$ $\&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_1 : \mathbf{float}[B_2; V_2]]\!]\ \mathbf{then}$ $\mathcal{D}[\![\mathbf{uniform}(v_1, w_1) : \mathbf{float}[B; V]]\!]$ ... $\mathbf{else\ if}\ v_1' ==_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_i : \mathbf{float}[B_1; V_1]]\!]$ $\&\&\ v_2' ==_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_j : \mathbf{float}[B_2; V_2]]\!]\ \mathbf{then}$ $\mathcal{D}[\![\mathbf{uniform}(v_i, w_j) : \mathbf{float}[B; V]]\!]$ ... $\mathbf{else}\ \mathcal{D}[\![\mathbf{uniform}(v_n, w_m) : \mathbf{float}[B; V]]\!]$

Fig. 13. Discretization function, where $\text{CDF}(b_0) = 0$ and $\text{CDF}(b_{n+1}) = 1$. For the **uniform** distribution, $V = \top$ is implicit, and all remaining cut-set and value-set cases are ruled out by the constraints enforced during type inference. In particular, dispatchability ensures that the finite value sets enumerated by the last case are nonempty. For all other language constructs, $\mathcal{D}[\![\cdot]\!]$ recurses structurally over them.

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \mathbf{fin}(\lvert B\rvert + 1) & \text{if } \tau = \mathbf{float}[B; V] \text{ and } B \neq \top \\ \mathbf{float}[B; V] & \text{if } \tau = \mathbf{float}[B; V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 : \tau}$$

$$\textsc{Discrete}\ \frac{}{\Gamma \vdash \textbf{discrete}(p_0, \ldots, p_{n-1}) : \textbf{fin}(n)} \qquad \textsc{LessEq}\ \frac{\Gamma \vdash e : \textbf{int}}{\Gamma \vdash e \le i : \textbf{bool}} \qquad \textsc{Pair}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 * \tau_2}$$

$$\textsc{Fst}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \textbf{fst}\ e : \tau_1} \qquad \textsc{Snd}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \textbf{snd}\ e : \tau_2} \qquad \textsc{Fun}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \textbf{fun}\ x\ \rightarrow\ e : \tau_1 \rightarrow \tau_2}$$

$$\textsc{App}\ \frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2} \qquad \textsc{True}\ \frac{}{\Gamma \vdash \text{true} : \textbf{bool}} \qquad \textsc{False}\ \frac{}{\Gamma \vdash \text{false} : \textbf{bool}}$$

$$\textsc{And}\ \frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \textbf{bool}}{\Gamma \vdash e_1\ \&\&\ e_2 : \textbf{bool}} \qquad \textsc{Or}\ \frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \textbf{bool}}{\Gamma \vdash e_1\ ||\ e_2 : \textbf{bool}} \qquad \textsc{Not}\ \frac{\Gamma \vdash e : \textbf{bool}}{\Gamma \vdash \text{not}\ e : \textbf{bool}}$$

$$\textsc{Unit}\ \frac{}{\Gamma \vdash () : \textbf{unit}} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \textbf{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \textbf{bool}}$$

$$\textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \textbf{bool}} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e : \textbf{bool}}{\Gamma \vdash \textbf{observe}\ e : \textbf{unit}} \qquad \textsc{Seq}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1; e_2 : \tau_2}$$

$$\textsc{Fix}\ \frac{\Gamma, f : \tau_1 \rightarrow \tau_2, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fix}\ f\ x := e : \tau_1 \rightarrow \tau_2} \qquad \textsc{Nil}\ \frac{}{\Gamma \vdash \text{nil} : \textbf{list}(\tau)} \qquad \textsc{Cons}\ \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \textbf{list}(\tau)}{\Gamma \vdash e_1 :: e_2 : \textbf{list}(\tau)}$$

$$\textsc{Match}\ \frac{\Gamma \vdash e : \textbf{list}(\tau_1) \qquad \Gamma \vdash e_1 : \tau_2 \qquad \Gamma, h : \tau_1, t : \textbf{list}(\tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \text{match}\ e\ \text{with nil} \rightarrow e_1 \mid h :: t \rightarrow e_2\ \text{end} : \tau_2}$$

$$\textsc{Diverge}\ \frac{}{\Gamma \vdash \textbf{diverge} : \tau}$$

Fig. 25. Typing rules for general language constructs

Case $e = (e_1 : \textbf{float}[B; V_1] \le e_2 : \textbf{float}[B; V_2])$. Analogous to the $<$ case, using answers$_{\le}(B, V_1, V_2)$ and FinLessEq.

Case $e = (\textbf{uniform}(e_1 : \textbf{float}[B_1; V_1],\ e_2 : \textbf{float}[B_2; V_2]) : \textbf{float}[B; \top])$. The Continuous-2 rule was applied, so recovers(B_i, V_i) and dispatchable(B_i, V_i) hold for $i \in \{1, 2\}$. Hence, whenever

$B_i \neq \top$, dispatchability gives $V_i \notin \{\emptyset, \top\}$. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \mathbf{float}[B_i; V_i]]\!] : \mathcal{D}[\![\mathbf{float}[B_i; V_i]]\!]$ for $i = 1, 2$. We consider each sub-case of the discretization function:

- $e_1, e_2 \in \mathbb{R}$ and $e_1 > e_2$: then $\mathcal{D}[\![\mathbf{uniform}(\ldots) : \mathbf{float}[B; \top]]\!] = \mathbf{diverge}$, which is well-typed at any type by Diverge.
- $B = \top$: then $\mathcal{D}[\![\mathbf{float}[B; \top]]\!] = \mathbf{float}$, and

$$\mathcal{D}[\![\mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]]\!] = \mathbf{uniform}(\mathcal{D}[\![e_1 : \mathbf{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \mathbf{float}[B_2; V_2]]\!]).$$

  Since recovers(B_1, V_1) and recovers(B_2, V_2) are preserved under discretization, the result is well-typed at $\mathbf{float}$ by Continuous-2.
- $B \neq \top$ and $e_1, e_2 \in \mathbb{R}$: then $\mathcal{D}[\![\mathbf{float}[B; \top]]\!] = \mathbf{fin}(|B| + 1)$, and

$$\mathcal{D}[\![\mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]]\!] = \mathbf{discrete}(p_0, \ldots, p_{|B|})$$

  where $p_k = \mathrm{CDF}(b_{k+1}) - \mathrm{CDF}(b_k)$. Each $p_k \in [0, 1]$ and $\sum_k p_k = 1$, so this is well-typed at $\mathbf{fin}(|B| + 1)$ by Discrete.
- $B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, and e_1 or e_2 is a variable: the algorithm generates conditionals over all pairs (v_i, w_j), each branch dispatching to a constant parameter in $\mathbf{uniform}$. The value sets V_1 and V_2 are nonempty by the preceding dispatchability argument, so this enumeration has at least one pair and its final else-branch is defined. By the previous sub-case, each branch $\mathcal{D}[\![\mathbf{uniform}(v_i, w_j) : \mathbf{float}[B; \top]]\!]$ is well-typed at $\mathbf{fin}(|B| + 1) = \mathcal{D}[\![\mathbf{float}[B; \top]]\!]$. The conditional structure is well-typed by If (as in the $\mathbf{if}$ case above), so the full expression is well-typed at $\mathcal{D}[\![\mathbf{float}[B; \top]]\!]$.

Case e is any other language construct (functions, pairs, lists, *etc.*). By definition of $\mathcal{D}[\![\cdot]\!]$, the discretization recurses structurally over e and τ. That is, $\mathcal{D}[\![e : \tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each subexpression and subtype of $e : \tau$, and $\mathcal{D}[\![\tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each component type of τ. The result then follows immediately by applying the inductive hypothesis to each subterm, analogous to the above cases.

In every case, the typing derivation is preserved under $\mathcal{D}[\![\cdot]\!]$. Therefore,

$$\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]. \qquad \square$$

F Soundness Proof

F.1 Measure Theory Preliminaries

We first give a brief overview of basic measure-theoretic terms and lemmas, and refer the reader to standard textbooks for further background [43, 44].

Topological space. A topological space is a pair $(X, \mathcal{T})$ where $\mathcal{T}$ is a collection of subsets of X, called open sets, such that $\emptyset, X \in \mathcal{T}$, and $\mathcal{T}$ is closed under arbitrary unions and finite intersections.

σ-algebra. A σ-algebra on a set X is a collection Σ of subsets of X that includes X itself and is closed under complement and under countable unions. By implication, a σ-algebra includes the empty set $\emptyset$ and is closed under countable intersections. Given a topological space $(X, \mathcal{T})$, the Borel σ-algebra $\mathcal{B}(X)$ is the smallest σ-algebra containing all open sets in $\mathcal{T}$.

Measurable space. A measurable space is a pair (X, Σ) consisting of a set X and a σ-algebra Σ on X. When Σ is the Borel σ-algebra on a topological space X, we call (X, Σ) a Borel measurable space.

Measure. Let (X, Σ) be a measurable space. A function $\mu : \Sigma \to [0, \infty]$ is called a measure if it satisfies the following properties: nonnegativity, $\mu(\emptyset) = 0$, and countable additivity. When Σ is a Borel σ-algebra, μ is called a Borel measure.

$\mathbf{e} : \boldsymbol{\tau}$	**constraints**	$[\![\boldsymbol{e} : \boldsymbol{\tau}]\!]$
$\mathtt{diverge} : \tau$		$\lambda A.0$
$v : \tau$		$\delta_{v:\tau}$
$\mathtt{let}\ x = e_1 : \tau_1\ \mathtt{in}\ e_2 : \tau_2 : \tau_2$	$e_1 = v$ e_1 no val.	$\delta_{e_2[v/x]:\tau_2}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathtt{let}\ x=g\ \mathtt{in}\ e_2:\tau_2:\tau_2}$
$\mathtt{if}\ e_1 : \mathtt{bool}\ \mathtt{then}\ e_2 : \tau\ \mathtt{else}\ e_3 : \tau : \tau$	$e_1 = \text{true}$ $e_1 = \text{false}$ o.w.	$\delta_{e_2:\tau}$ $\delta_{e_3:\tau}$ $[\![e_1 : \mathtt{bool}]\!] \gg\!= \lambda g.\delta_{\mathtt{if}\ g\ \mathtt{then}\ e_2:\tau\ \mathtt{else}\ e_3:\tau:\tau}$
$\mathtt{observe}\ (e : \mathtt{bool}) : \mathtt{unit}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{\mathtt{unit}}$ $\delta_{\bot}$ $[\![e : \mathtt{bool}]\!] \gg\!= \lambda g.\delta_{\mathtt{observe}\ (g:\mathtt{bool}):\mathtt{unit}}$
$e_1 : \tau_1 < e_2 : \tau_2 : \mathtt{bool}$ for $\tau_1 = \mathtt{float}[B; V_1], \tau_2 = \mathtt{float}[B; V_2]$ or $\tau_1 = \tau_2 = \mathtt{fin}(n)$	$e_1 = v_1, e_2 = v_2, v_1 < v_2$ $e_1 = v_1, e_2 = v_2, v_1 \geq v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{\text{true}:\mathtt{bool}}$ $\delta_{\text{false}:\mathtt{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{v_1:\tau_1<g:\mathtt{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g<e_2:\tau_2:\mathtt{bool}}$
$\mathtt{uniform}(e_1 : \tau_1, e_2 : \tau_2) : \mathtt{float}[B; V]$	$e_1 = v_1, e_2 = v_2, v_1 \leq v_2$ $e_1 = v_1, e_2 = v_2, v_1 > v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\text{Uniform}(v_1, v_2) \gg\!= \lambda v.\delta_{(v:\mathtt{float}[B;V])}$ $[\![\mathtt{diverge} : \tau]\!]$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{\mathtt{uniform}(v_1:\tau_1,g):\tau}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathtt{uniform}(g,e_2:\tau_2):\tau}$
$\mathtt{discrete}(v_0, \ldots, v_{n-1}) : \mathtt{fin}(n)$	$\sum_i v_i = 1$	$\text{Discrete}(v_0, \ldots, v_{n-1})$
$\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(e_1 : \tau_1, e_2 : \tau_2) : \mathtt{fin}(\lvert B\rvert + 1)$ $\tau = \mathtt{fin}(\lvert B\rvert + 1)$, $\tau_1 = \mathtt{fin}(\lvert B_1\rvert + 1), \tau_2 = \mathtt{fin}(\lvert B_2\rvert + 1)$, $V_1 = \{v_1, \ldots, v_n\}, V_2 = \{w_1, \ldots, w_m\}, n, m \geq 1$	$e_1 = \mathcal{D}[\![v_i : \mathtt{float}[B_1; V_1]]\!]$, $e_2 = \mathcal{D}[\![w_j : \mathtt{float}[B_2; V_2]]\!]$ $e_1 = v', e_2 \neq w'$ o.w.	$[\![\mathcal{D}[\![\mathtt{uniform}(v_i, w_j) : \mathtt{float}[B; \top]]\!] : \tau]\!]$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(v':\tau_1,g)}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(g,e_2:\tau_2)}$

Table 1. Small-step operational semantics. In the last case, the existence of suitable v_i and w_j is guaranteed since $\text{recovers}(B_i, V_i)$ and $e_i \in V_i$ for $i \in \{1, 2\}$. Here, every function of the form $f(g) = \lambda g.\delta_{\ldots}$ is extended to $f_\bot$ in the sense of Definition F.29.

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a Markov kernel. □

Lemma F.33 (Composition of Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \textit{and} \quad Q : Y \to \mathcal{D}(Z)$$

are Markov kernels, then their composition

$$(P \gg\!= Q)(A) \;:=\; \int_{x \in supp(P)} Q(x)(A)\, P(dx)$$

defines a Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

F.4 Small-Step Semantics

Throughout this section, we denote by $e : \tau$ a well-typed expression of type τ and implicitly assume that all sub-expressions are annotated with types, as well. Moreover, for the typed expression

$e : \tau$	constraints	$[\![e : \tau]\!]$
$e_1 : \tau_1$ && $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = b_1 = \text{true}, e_2 = b_2 = \text{true}$ $e_1 = b_1, e_2 = b_2$ $e_1 = b_1, e_2 \neq b_2$ o.w.	$\delta_{\text{true}:\mathbf{bool}}$ $\delta_{\text{false}:\mathbf{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{b_1:\tau_1 \&\& g:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \&\& e_2:\tau_2:\mathbf{bool}}$
$e_1 : \tau_1$ || $e_2 : \tau_2 : \mathbf{bool}$ $\tau_1 = \mathbf{bool}, \tau_2 = \mathbf{bool}$	$e_1 = b_1 = \text{false}, e_2 = b_2 = \text{false}$ $e_1 = b_1, e_2 = b_2$ $e_1 = b_1, e_2 \neq b_2$ o.w.	$\delta_{\text{false}:\mathbf{bool}}$ $\delta_{\text{true}:\mathbf{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{b_1:\tau_1 \,\|\| g:\mathbf{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \,\|\| e_2:\tau_2:\mathbf{bool}}$
not $e : \tau : \mathbf{bool}$ $\tau = \mathbf{bool}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{\text{false}:\mathbf{bool}}$ $\delta_{\text{true}:\mathbf{bool}}$ $[\![e : \tau]\!] \gg\!= \lambda g.\delta_{\text{not } g:\mathbf{bool}}$
$(e_1 : \tau_1, e_2 : \tau_2) : \tau_1 * \tau_2$	$e_1 = v_1, e_2 = v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{(v_1,v_2):\tau_1*\tau_2}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{(v_1:\tau_1,f):\tau_1*\tau_2}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{(f,e_2:\tau_2):\tau_1*\tau_2}$
$\mathbf{fst}\ (e : \tau_1 * \tau_2) : \tau_1$	$e = (v_1, v_2)$ o.w.	$\delta_{v_1:\tau_1}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{fst}\ e:\tau_1}$
$\mathbf{snd}\ (e : \tau_1 * \tau_2) : \tau_2$	$e = (v_1, v_2)$ o.w.	$\delta_{v_2:\tau_2}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\mathbf{snd}\ e:\tau_2}$
$(e_1 : \tau_1 \to \tau_2\ e_2 : \tau_1) : \tau_2$	$e_1 = \mathbf{fun}\ x \to e$ or $\mathbf{fix}\ g\ x := e, e_2 = v_2$ $e_1 = \mathbf{fun}\ x \to e$ or $\mathbf{fix}\ g\ x := e, e_2 \neq v_2$ o.w.	$\delta_{e[v_2/x]:\tau_2}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{e_1:\tau_1 \to \tau_2\ g:\tau_2}$ $[\![e_1 : \tau_1 \to \tau_2]\!] \gg\!= \lambda g.\delta_{f\ e_2:\tau_1:\tau_2}$
$\text{nil} : \mathbf{list}(\tau)$		$\delta_{\text{nil}:\mathbf{list}(\tau)}$
$(e_1 : \tau_1 :: e_2 : \mathbf{list}(\tau)) : \mathbf{list}(\tau)$	$e_1 = v_1, e_2 = v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{v_1::v_2:\mathbf{list}(\tau)}$ $[\![e_2 : \mathbf{list}(\tau)]\!] \gg\!= \lambda g.\delta_{v_1:\tau_1::g:\mathbf{list}(\tau)}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{f::e_2:\mathbf{list}(\tau):\mathbf{list}(\tau)}$
$(\mathbf{match}\ e : \mathbf{list}(\tau_1)\ \mathbf{with}$ $\quad \mid \text{nil} \to e_1 : \tau_2$ $\quad \mid h :: t \to e_2 : \tau_2$ $\quad \mathbf{end}) : \tau_2$	$e = \text{nil}$ $e = v_h :: v_t$ o.w.	$\delta_{e_1:\tau_2}$ $\delta_{e_2[v_h/h,v_t/t]:\tau_2}$ $[\![e : \mathbf{list}(\tau_1)]\!] \gg\!= \lambda g.\delta_{\mathbf{match}\ g\ \mathbf{with}\ldots:\tau_2}$

Table 2. Small-step operational semantics (continued). Here, every function of the form $f(g) = \lambda g.\delta_{\ldots}$ is extended to $f_\perp$ in the sense of Definition F.29.

$e : \tau = \mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1], e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B, V]$ with $B, B_1, B_2 \neq \top$ and $V_1, V_2 \notin \{\mathbb{0}, \top\}$, we introduce an auxiliary statement $\mathsf{DC}_{B,B_1,B_2,V_1,V_2}$ and let

$$\mathcal{D}[\![e : \tau]\!] = \mathsf{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![e_1 : \mathbf{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \mathbf{float}[B_2; V_2]]\!])\ .$$

We define the semantics of $\mathsf{DC}_{B,B_1,B_2,V_1,V_2}$ in such a way that it behaves like the discretized program in the last case of Figure 13 but resolves the let-bindings and the conditional choices in a single step, which will simplify our proof. We equip this statement with the following typing rule :

$$\frac{e_1 : \mathbf{fin}(|B_1| + 1) \qquad e_2 : \mathbf{fin}(|B_2| + 1) \qquad \text{recovers}(B_1, V_1) \qquad \text{recovers}(B_2, V_2)}{\mathsf{DC}_{B,B_1,B_2,V_1,V_2}(e_1, e_2) : \mathbf{fin}(|B| + 1)}\ \textsc{TDiscreteCase}$$

Equipped with these notions, we define the small-step semantics of our language:

Definition F.34. The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \perp)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.